```
            return (long)Math.ceil(dayDiff);
        }

        public static int getMonthsDiffAbove(String beginDate, String endDate) throws
IllegalArgumentException {
            return getMonthsDiffAbove(toDate(beginDate), toDate(endDate));
        }

        public static int getMonthsDiffAbove(Date beginDate, Date endDate) {
            if (endDate.before(beginDate))
                return -doGetMonthsDiffAbove(endDate, beginDate);
            return doGetMonthsDiffAbove(beginDate, endDate);
        }

        private static int doGetMonthsDiffAbove(Date beginDate, Date endDate) {
            CalendarWrapper begin = new CalendarWrapper(beginDate);
            CalendarWrapper end = new CalendarWrapper(endDate);
            int years = end.getYear() - begin.getYear();
            int months = end.getMonth() - begin.getMonth();
            int days = end.getDay() - begin.getDay();
            if (years == 0) {
                if (days == 0)
                return months;
                return months + ((days > 0) ? 1 : 0);
            }
            if (years > 0) {
                if (days == 0)
                return years * 12 + months;
                return years * 12 + months + ((days < 0) ? 0 : 1);
            }
            return years * 12 + months + ((days > 0) ? 1 : 0);
        }

        public static int getYearsDiffAbove(String beginDate, String endDate) throws
IllegalArgumentException {
            return getYearsDiffAbove(toDate(beginDate), toDate(endDate));
        }

        public static int getYearsDiffAbove(Date beginDate, Date endDate) {
            if (endDate.before(beginDate))
                return -doGetYearsDiffAbove(endDate, beginDate);
            return doGetYearsDiffAbove(beginDate, endDate);
        }

        private static int doGetYearsDiffAbove(Date beginDate, Date endDate) {
            int months = getMonthsDiffAbove(beginDate, endDate);
            return (int)Math.ceil(months / 12.0D);
        }

        public static String toLunarDate(String solarDate) throws IllegalArgumentException
{
            LunarCalendar lunarCal = new LunarCalendar();
            return String.valueOf(lunarCal.solar2lunar(solarDate)) +
(lunarCal.isLeapMonth() ? "(윤달)" : "");
        }

        public static String toSolarDate(String lunarDate, boolean leapMonth) throws
IllegalArgumentException {
            LunarCalendar lunarCal = new LunarCalendar();
            return lunarCal.lunar2solar(lunarDate, leapMonth);
        }
```

```
	public static int getQuater(String strDate) {
		String[] parsedDateArray = parseDate(strDate);
		if (parsedDateArray == null || parsedDateArray.length == 0)
			return -1;
		int month = Integer.parseInt(parsedDateArray[1]);
		return (month - 1) / 3 + 1;
	}

	public static int[] getDifference(String beginDate, String endDate) throws
IllegalArgumentException {
		if (!isValidDate(beginDate) || !isValidDate(endDate))
			throw new IllegalArgumentException("Date format error : " + endDate);
		return getDifference(toDate(beginDate), toDate(endDate));
	}

	public static int[] getDifference(Date beginDate, Date endDate) throws
IllegalArgumentException {
		CalendarWrapper begin = new CalendarWrapper(beginDate);
		CalendarWrapper end = new CalendarWrapper(endDate);
		if (endDate.before(beginDate)) {
			begin = new CalendarWrapper(endDate);
			end = new CalendarWrapper(beginDate);
		}else {
			begin = new CalendarWrapper(beginDate);
			end = new CalendarWrapper(endDate);
		}
		int months = Math.abs(getMonthsDiff(beginDate, endDate));
		int years = months / 12;
		months %= 12;
		int days = 0;
		if (begin.getDay() > end.getDay()) {
			days = (new CalendarWrapper(end.beforeMonth(1))).getLastDayOfMonth()
- begin.getDay() + end.getDay();
		}else {
			days = end.getDay() - begin.getDay();
		}
		return new int[] { years, months, days };
	}

	private static Date getSystemDate() {
		return new Date(System.currentTimeMillis());
	}

	private static String[] parseDate(String strDate) {
		if (strDate == null)
			return new String[0];
		String tempStrDate = strDate.replaceAll("^\\D+$", "");
		int len = tempStrDate.length();
		if (len >= 10)
			return new String[] { tempStrDate.substring(0, 4),
tempStrDate.substring(4, 6),
					tempStrDate.substring(6, 8), tempStrDate.substring(8, 10) };
		if (len >= 8)
			return new String[] { tempStrDate.substring(0, 4),
tempStrDate.substring(4, 6),
					tempStrDate.substring(6, 8), "00" };
		if (len >= 6)
			return new String[] { tempStrDate.substring(0, 4),
tempStrDate.substring(4, 6), "00", "00" };
		if (len >= 4)
			return new String[] { tempStrDate.substring(0, 4), "00", "00",
"00" };
```

```
			return new String[0];
		}

		public static String getCurrentDate(int format) {
			if (format >= dateFormatterList.length || format < 0)
				return null;
			SimpleDateFormat simpledateformat = new
SimpleDateFormat(dateFormatterList[format]);
			Calendar calendar = Calendar.getInstance();
			simpledateformat.setCalendar(calendar);
			String s =
simpledateformat.format(simpledateformat.getCalendar().getTime());
			return s;
		}

		public static String getDateDaysBefore(int before, String date) {
			return getBeforeDateString(date, 0, 0, before);
		}

		public static String getDate(String date, int format) {
			return getDate(Integer.parseInt(date.substring(0, 4)),
Integer.parseInt(date.substring(4, 6)), Integer.parseInt(date.substring(6)), format);
		}

		public static String getDate(int year, int month, int day, int format) {
			SimpleDateFormat simpledateformat = new
SimpleDateFormat(dateFormatterList[format]);
			Calendar calendar = Calendar.getInstance();
			calendar.set(year, month - 1, day);
			simpledateformat.setCalendar(calendar);
			String s =
simpledateformat.format(simpledateformat.getCalendar().getTime());
			return s;
		}
}
package nhnis.fw.commons.util;

import com.google.gson.Gson;

public class JsonUtil {

		public static final Gson GSON = new Gson();
}

package nhnis.fw.commons.util;

public class LunarCalendar {
		private final int[][] lunarMonthTable;

		private boolean isLunarLeapMonth;

		public LunarCalendar() {
			this.lunarMonthTable = new int[][] { { 2, 1, 2, 1, 2, 1, 2, 2, 1, 2, 1, 2 },
					{ 1, 2, 1, 1, 2, 1, 2, 5, 2, 2, 1, 2 }, { 1, 2, 1, 1, 2, 1,
2, 1, 2, 2, 2, 1 },
					{ 2, 1, 2, 1, 1, 2, 1, 2, 1, 2, 2, 2 }, { 1, 2, 1, 2, 3, 2,
1, 1, 2, 2, 1, 2 },
					{ 2, 2, 1, 2, 1, 1, 2, 1, 1, 2, 2, 1 }, { 2, 2, 1, 2, 2, 1,
1, 2, 1, 2, 1, 2 },
					{ 1, 2, 2, 4, 1, 2, 1, 2, 1, 2, 1, 2 }, { 1, 2, 1, 2, 1, 2,
2, 1, 2, 1, 2, 1 },
					{ 2, 1, 1, 2, 2, 1, 2, 1, 2, 2, 1, 2 }, { 1, 5, 1, 2, 1, 2,
```

1, 2, 2, 2, 1, 2 },
1, 2, 2, 1, 2, 2 },
2, 1, 1, 2, 1, 2 },
1, 2, 1, 2, 1, 2 },
1, 2, 2, 1, 2, 1 },
5, 2, 2, 1, 2, 2 },
1, 1, 2, 1, 2, 2 },
2, 1, 2, 1, 1, 2 },
2, 2, 1, 2, 1, 2 },
1, 2, 2, 1, 2, 2 },
1, 2, 1, 2, 2, 2 },
1, 1, 2, 1, 2, 1 },
2, 1, 2, 1, 1, 2 },
2, 2, 1, 2, 2, 1 },
2, 1, 2, 2, 2, 1 },
1, 2, 1, 2, 1, 2 },
1, 1, 2, 1, 2, 1 },
2, 1, 2, 2, 1, 2 },
1, 2, 2, 2, 1, 2 },
2, 1, 2, 1, 2, 2 },
3, 2, 1, 2, 1, 2 },
1, 2, 1, 2, 1, 2 },
1, 2, 2, 1, 2, 2 },
1, 2, 1, 2, 2, 2 },
1, 5, 2, 1, 2, 2 },
2, 1, 2, 1, 2, 1 },
2, 2, 1, 2, 1, 2 },
2, 1, 2, 2, 2, 1 },
2, 1, 1, 2, 2, 2 },
1, 2, 1, 2, 1, 2 },
2, 1, 2, 1, 2, 1 },

{ 1, 2, 1, 1, 2, 1, 2, 1, 2, 2, 2, 1 }, { 2, 1, 2, 1, 1, 5,
{ 2, 1, 2, 1, 1, 2, 1, 1, 2, 2, 1, 2 }, { 2, 2, 1, 2, 1, 1,
{ 2, 2, 1, 2, 5, 1, 2, 1, 2, 1, 1, 2 }, { 2, 1, 2, 2, 1, 2,
{ 1, 2, 1, 2, 1, 2, 2, 1, 2, 1, 2, 1 }, { 2, 3, 2, 1, 2, 2,
{ 2, 1, 1, 2, 1, 2, 1, 2, 2, 2, 1, 2 }, { 1, 2, 1, 1, 2, 1,
{ 1, 2, 1, 1, 2, 1, 1, 2, 2, 1, 2, 2 }, { 2, 1, 2, 1, 1, 2,
{ 2, 1, 2, 2, 3, 2, 1, 1, 2, 1, 2, 2 }, { 1, 2, 2, 1, 2, 1,
{ 2, 1, 2, 1, 2, 2, 1, 2, 1, 2, 1, 1 }, { 2, 1, 2, 5, 2, 1,
{ 1, 1, 2, 1, 2, 1, 2, 2, 1, 2, 2, 1 }, { 2, 1, 1, 2, 1, 2,
{ 1, 5, 1, 2, 1, 1, 2, 2, 1, 2, 2, 2 }, { 1, 2, 1, 1, 2, 1,
{ 1, 2, 2, 1, 1, 5, 1, 2, 1, 2, 2, 1 }, { 2, 2, 2, 1, 1, 2,
{ 2, 2, 2, 1, 2, 1, 2, 1, 1, 2, 1, 2 }, { 1, 2, 2, 1, 6, 1,
{ 1, 2, 1, 2, 2, 1, 2, 2, 1, 2, 1, 2 }, { 1, 1, 2, 1, 2, 1,
{ 2, 1, 4, 1, 2, 1, 2, 1, 2, 2, 2, 1 }, { 2, 1, 1, 2, 1, 1,
{ 2, 2, 1, 1, 2, 1, 4, 1, 2, 2, 1, 2 }, { 2, 2, 1, 1, 2, 1,
{ 2, 2, 1, 2, 1, 2, 1, 1, 2, 1, 2, 1 }, { 2, 2, 1, 2, 2, 4,
{ 2, 1, 2, 2, 1, 2, 2, 1, 2, 1, 1, 2 }, { 1, 2, 1, 2, 1, 2,
{ 1, 1, 2, 4, 1, 2, 1, 2, 2, 1, 2, 2 }, { 1, 1, 2, 1, 1, 2,
{ 2, 1, 1, 2, 1, 1, 2, 1, 2, 2, 1, 2 }, { 2, 5, 1, 2, 1, 1,
{ 2, 1, 2, 1, 2, 1, 1, 2, 1, 2, 1, 2 }, { 2, 2, 1, 2, 1, 2,
{ 2, 1, 2, 2, 1, 2, 1, 1, 2, 1, 2, 1 }, { 2, 1, 2, 2, 1, 2,
{ 1, 2, 1, 2, 4, 2, 1, 2, 1, 2, 1, 2 }, { 1, 2, 1, 1, 2, 2,
{ 1, 1, 2, 1, 1, 2, 1, 2, 2, 1, 2, 2 }, { 2, 1, 4, 1, 1, 2,
{ 1, 2, 1, 2, 1, 1, 2, 1, 2, 1, 2, 2 }, { 2, 1, 2, 1, 2, 1,
{ 1, 2, 2, 1, 2, 1, 1, 2, 1, 2, 1, 2 }, { 1, 2, 2, 1, 2, 1,
{ 2, 1, 2, 1, 2, 5, 2, 1, 2, 1, 2, 1 }, { 2, 1, 2, 1, 2, 1,
{ 1, 2, 1, 1, 2, 1, 2, 2, 1, 2, 2, 1 }, { 2, 1, 2, 3, 2, 1,
{ 2, 1, 2, 1, 1, 2, 1, 2, 1, 2, 2, 2 }, { 1, 2, 1, 2, 1, 1,
{ 1, 2, 5, 2, 1, 1, 2, 1, 1, 2, 2, 1 }, { 2, 2, 1, 2, 2, 1,
{ 1, 2, 2, 1, 2, 1, 5, 2, 1, 2, 1, 2 }, { 1, 2, 1, 2, 1, 2,
{ 2, 1, 1, 2, 2, 1, 2, 1, 2, 2, 1, 2 }, { 1, 2, 1, 1, 5, 2,

1, 2, 2, 2, 1, 2 },
1, 1, 2, 2, 2, 1 },
2, 1, 1, 2, 1, 2 },
1, 2, 1, 2, 1, 1 },
1, 2, 2, 1, 2, 1 },
1, 2, 2, 1, 2, 2 },
1, 1, 2, 1, 2, 2 },
2, 1, 1, 2, 1, 2 },
2, 2, 1, 2, 1, 2 },
1, 2, 2, 1, 2, 2 },
1, 2, 1, 2, 2, 2 },
1, 1, 2, 1, 2, 1 },
1, 5, 2, 1, 1, 2 },
2, 2, 1, 2, 2, 1 },
2, 1, 2, 2, 2, 1 },
1, 2, 1, 2, 1, 2 },
2, 1, 1, 2, 1, 2 },
2, 1, 2, 2, 1, 1 },
1, 2, 2, 2, 1, 2 },
2, 1, 2, 1, 2, 2 },
1, 1, 2, 1, 2, 1 },
1, 2, 1, 2, 1, 2 },
2, 2, 2, 1, 2, 1 },
1, 2, 1, 2, 2, 2 },
1, 2, 1, 2, 1, 2 },
2, 1, 2, 1, 2, 1 },
2, 2, 1, 2, 1, 2 },
2, 1, 2, 2, 2, 1 },
2, 1, 1, 2, 2, 2 },
1, 2, 1, 1, 2, 2 },
2, 1, 2, 1, 2, 1 },

{ 1, 2, 1, 1, 2, 1, 2, 1, 2, 2, 2, 1 }, { 2, 1, 2, 1, 1, 2,
{ 2, 2, 1, 5, 1, 2, 1, 1, 2, 2, 1, 2 }, { 2, 2, 1, 2, 1, 1,
{ 2, 2, 1, 2, 1, 2, 1, 5, 2, 1, 1, 2 }, { 2, 1, 2, 2, 1, 2,
{ 2, 2, 1, 2, 1, 2, 2, 1, 2, 1, 2, 1 }, { 2, 1, 1, 2, 1, 6,
{ 2, 1, 1, 2, 1, 2, 1, 2, 2, 1, 2, 2 }, { 1, 2, 1, 1, 2, 1,
{ 2, 1, 2, 3, 2, 1, 1, 2, 2, 1, 2, 2 }, { 2, 1, 2, 1, 1, 2,
{ 2, 1, 2, 2, 1, 1, 2, 1, 1, 5, 2, 2 }, { 1, 2, 2, 1, 2, 1,
{ 1, 2, 2, 1, 2, 2, 1, 2, 1, 2, 1, 1 }, { 2, 1, 2, 2, 1, 5,
{ 1, 1, 2, 1, 2, 1, 2, 2, 1, 2, 2, 1 }, { 2, 1, 1, 2, 1, 2,
{ 1, 2, 1, 1, 5, 1, 2, 1, 2, 2, 2, 2 }, { 1, 2, 1, 1, 2, 1,
{ 1, 2, 2, 1, 1, 2, 1, 1, 2, 1, 2, 2 }, { 1, 2, 5, 2, 1, 2,
{ 2, 2, 2, 1, 2, 1, 2, 1, 1, 2, 1, 2 }, { 1, 2, 2, 1, 2, 2,
{ 1, 2, 1, 2, 2, 1, 2, 1, 2, 2, 1, 2 }, { 1, 1, 2, 1, 2, 1,
{ 2, 1, 1, 2, 3, 2, 2, 1, 2, 2, 2, 1 }, { 2, 1, 1, 2, 1, 1,
{ 2, 2, 1, 1, 2, 1, 1, 2, 1, 2, 2, 1 }, { 2, 2, 2, 3, 2, 1,
{ 2, 2, 1, 2, 1, 2, 1, 1, 2, 1, 2, 1 }, { 2, 2, 1, 2, 2, 1,
{ 1, 5, 2, 2, 1, 2, 1, 2, 1, 2, 1, 2 }, { 1, 2, 1, 2, 1, 2,
{ 2, 1, 2, 1, 2, 1, 5, 2, 2, 1, 2, 2 }, { 1, 1, 2, 1, 1, 2,
{ 2, 1, 1, 2, 1, 1, 2, 1, 2, 2, 1, 2 }, { 2, 2, 1, 1, 5, 1,
{ 2, 1, 2, 1, 2, 1, 1, 2, 1, 2, 1, 2 }, { 2, 1, 2, 2, 1, 2,
{ 2, 1, 6, 2, 1, 2, 1, 1, 2, 1, 2, 1 }, { 2, 1, 2, 2, 1, 2,
{ 1, 2, 1, 2, 1, 2, 1, 2, 5, 2, 1, 2 }, { 1, 2, 1, 1, 2, 1,
{ 2, 1, 2, 1, 1, 2, 1, 2, 2, 1, 2, 2 }, { 2, 1, 1, 2, 3, 2,
{ 1, 2, 1, 2, 1, 1, 2, 1, 2, 1, 2, 2 }, { 2, 1, 2, 1, 2, 1,
{ 2, 1, 2, 5, 2, 1, 1, 2, 1, 2, 1, 2 }, { 1, 2, 2, 1, 2, 1,
{ 2, 1, 2, 1, 2, 2, 1, 2, 1, 2, 1, 2 }, { 1, 5, 2, 1, 2, 1,
{ 1, 2, 1, 1, 2, 1, 2, 2, 1, 2, 2, 1 }, { 2, 1, 2, 1, 1, 5,
{ 2, 1, 2, 1, 1, 2, 1, 2, 1, 2, 2, 2 }, { 1, 2, 1, 2, 1, 1,
{ 1, 2, 2, 1, 5, 1, 2, 1, 1, 2, 2, 1 }, { 2, 2, 1, 2, 2, 1,
{ 1, 2, 1, 2, 2, 1, 2, 1, 2, 1, 2, 1 }, { 2, 1, 5, 2, 1, 2,
{ 2, 1, 1, 2, 1, 2, 2, 1, 2, 2, 1, 2 }, { 1, 2, 1, 1, 2, 1,

```
2, 1, 2, 2, 5, 2 },
                        { 1, 2, 1, 1, 2, 1, 2, 1, 2, 2, 2, 1 }, { 2, 1, 2, 1, 1, 2,
1, 1, 2, 2, 1, 2 },
                        { 2, 2, 1, 2, 1, 4, 1, 1, 2, 2, 1, 2 }, { 2, 2, 1, 2, 1, 1,
2, 1, 1, 2, 1, 2 },
                        { 2, 2, 1, 2, 1, 2, 1, 2, 1, 1, 2, 1 }, { 2, 2, 1, 2, 5, 2,
1, 2, 1, 2, 1, 1 },
                        { 2, 1, 2, 2, 1, 2, 2, 1, 2, 1, 2, 1 }, { 2, 1, 1, 2, 1, 2,
2, 1, 2, 2, 1, 2 },
                        { 1, 5, 1, 2, 1, 2, 1, 2, 2, 2, 1, 2 }, { 1, 2, 1, 1, 2, 1,
1, 2, 2, 1, 2, 2 } };
        }

        public enum LUNAR_SOLAR {
                TO_SOLAR, TO_LUNAR;
        }

        public String solar2lunar(String solarDateStr) throws IllegalArgumentException {
                String tempSolarDateStr = solarDateStr.replaceAll("\\D", "");
                int year = Integer.parseInt(tempSolarDateStr.substring(0, 4));
                int month = Integer.parseInt(tempSolarDateStr.substring(4, 6));
                int day = Integer.parseInt(tempSolarDateStr.substring(6, 8));
                return lunarCalc(year, month, day, LUNAR_SOLAR.TO_LUNAR, false);
        }

        public String lunar2solar(String lunarDateStr, boolean isLunarLeapMonth) throws
IllegalArgumentException {
                String tempLunarDateStr = lunarDateStr.replaceAll("\\D", "");
                int year = Integer.parseInt(tempLunarDateStr.substring(0, 4));
                int month = Integer.parseInt(tempLunarDateStr.substring(4, 6));
                int day = Integer.parseInt(tempLunarDateStr.substring(6, 8));
                return lunarCalc(year, month, day, LUNAR_SOLAR.TO_SOLAR, isLunarLeapMonth);
        }

        public String solar2lunar(int year, int month, int day) throws
IllegalArgumentException {
                return lunarCalc(year, month, day, LUNAR_SOLAR.TO_LUNAR, false);
        }

        public String lunar2solar(int year, int month, int day, boolean isLunarLeapMonth)
throws IllegalArgumentException {
                return lunarCalc(year, month, day, LUNAR_SOLAR.TO_SOLAR, isLunarLeapMonth);
        }

        public String lunarCalc(int year, int month, int day, LUNAR_SOLAR type, boolean
isLeapMonth)
                        throws IllegalArgumentException {
                int solYear = 0;
                int solMonth = 0;
                int solDay = 0;
                int lunYear = 0;
                int lunMonth = 0;
                int lunDay = 0;
                if (year < 1900 || year > 2040)
                        throw new IllegalArgumentException("Unsupported year range : " +
year);
                int[] solMonthDay = { 31, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
                if (year >= 2000) {
                        solYear = 2000;
                        solMonth = 1;
                        solDay = 1;
                        lunYear = 1999;
```

```
                lunMonth = 11;
                lunDay = 25;
                this.isLunarLeapMonth = false;
                solMonthDay[1] = 29;
            } else if (year >= 1970) {
                solYear = 1970;
                solMonth = 1;
                solDay = 1;
                lunYear = 1969;
                lunMonth = 11;
                lunDay = 24;
                this.isLunarLeapMonth = false;
                solMonthDay[1] = 28;
            } else if (year >= 1940) {
                solYear = 1940;
                solMonth = 1;
                solDay = 1;
                lunYear = 1939;
                lunMonth = 11;
                lunDay = 22;
                this.isLunarLeapMonth = false;
                solMonthDay[1] = 29;
            } else {
                solYear = 1900;
                solMonth = 1;
                solDay = 1;
                lunYear = 1899;
                lunMonth = 12;
                lunDay = 1;
                this.isLunarLeapMonth = false;
                solMonthDay[1] = 28;
            }
            int lunIndex = lunYear - 1899;
            do {
                byte b = 0;
                if (type.equals(LUNAR_SOLAR.TO_LUNAR) && year == solYear && month ==
solMonth && day == solDay)
                    return String.format("%04d%02d%02d",
                            new Object[] { Integer.valueOf(lunYear),
Integer.valueOf(lunMonth), Integer.valueOf(lunDay) });
                if (type.equals(LUNAR_SOLAR.TO_SOLAR) && year == lunYear && month ==
lunMonth && day == lunDay
                        && isLeapMonth == this.isLunarLeapMonth)
                    return String.format("%04d%02d%02d",
                            new Object[] { Integer.valueOf(solYear),
Integer.valueOf(solMonth), Integer.valueOf(solDay) });
                if (solMonth == 12 && solDay == 31) {
                    solYear++;
                    solMonth = 1;
                    solDay = 1;
                    if (solYear % 400 == 0) {
                        solMonthDay[1] = 29;
                    } else if (solYear % 100 == 0) {
                        solMonthDay[1] = 28;
                    } else if (solYear % 4 == 0) {
                        solMonthDay[1] = 29;
                    } else {
                        solMonthDay[1] = 28;
                    }
                } else if (solMonthDay[solMonth - 1] == solDay) {
                    solMonth++;
                    solDay = 1;
```

```
                        } else {
                                solDay++;
                        }
                        if (lunMonth == 12 && ((this.lunarMonthTable[lunIndex][lunMonth - 1]
== 1 && lunDay == 29)
                                        || (this.lunarMonthTable[lunIndex][lunMonth - 1] == 2
&& lunDay == 30))) {
                                lunYear++;
                                lunMonth = 1;
                                lunDay = 1;
                                lunIndex = lunYear - 1899;
                                if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 1) {
                                        b = 29;
                                } else if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 2)
{
                                        b = 30;
                                }
                        } else if (lunDay == b) {
                                if (this.lunarMonthTable[lunIndex][lunMonth - 1] >= 3
&& !this.isLunarLeapMonth) {
                                        lunDay = 1;
                                        this.isLunarLeapMonth = true;
                                } else {
                                        lunMonth++;
                                        lunDay = 1;
                                        this.isLunarLeapMonth = false;
                                }
                                if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 1) {
                                        b = 29;
                                } else if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 2)
{
                                        b = 30;
                                } else if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 3)
{
                                        b = 29;
                                } else if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 4
&& !this.isLunarLeapMonth) {
                                        b = 29;
                                } else if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 4
&& this.isLunarLeapMonth) {
                                        b = 30;
                                } else if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 5
&& !this.isLunarLeapMonth) {
                                        b = 30;
                                } else if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 5
&& this.isLunarLeapMonth) {
                                        b = 29;
                                } else if (this.lunarMonthTable[lunIndex][lunMonth - 1] == 6)
{
                                        b = 30;
                                }
                        } else {
                                lunDay++;
                        }
                } while (!type.equals(LUNAR_SOLAR.TO_SOLAR) || !isLeapMonth || lunYear <=
year || lunMonth <= month);
                throw new IllegalArgumentException("Input date  is not a leap year : " +
year);
        }

        public boolean isLeapMonth() {
                return this.isLunarLeapMonth;
```